Problem-Sized Lean Worlds

How AI-assisted research is recorded, checked, and published

Will Cook*

September 2026

Abstract

Plectis is a research environment for studying open mathematical problems with AI agents. It records the statements already proved, the approaches that failed, and the questions still worth pursuing, together with the source material needed to examine them. The aim is to let a new research attempt begin where the previous work stopped.

This paper describes the implementation. Agents retrieve a small relevant part of the research record, save their proof attempts and check results, and update the formal source and papers when a result changes. A public Git repository makes the published work available for others to read, reproduce, and extend. We explain the design through a failed extension of a multiplication identity and a finite certificate whose scope must be preserved in publication. The contribution is the integration of these research and publication operations in a working prototype. It covers eight Erdős problems, all still open. The implementation and author-operated checks are available; whether this organisation improves research productivity has yet to be tested with outside contributors.

1 The problem: many kinds of evidence

A researcher returning to a difficult problem needs to know what has changed. Which statements have been proved? Why did the last approach stop? Has someone already tried the calculation now being proposed? A finished paper answers some of these questions, but much of the work can disappear between attempts. That loss is particularly easy when an AI agent's context ends and another agent starts from a summary.

We built Plectis to keep this work available. Each problem has a versioned record containing its literature, formal statements, computations, unsuccessful approaches, and papers. An agent can query that record, investigate a smaller question, and save a result that another researcher can inspect. The public repository contains the published mathematics and the tools for continuing it; the private workbench coordinates the agents that produced the initial work. The companion strategy paper explains how outside contributions return through GitHub with evidence and credit.

What this paper contributes. We describe how the implementation connects research records to formal checks and public explanations. Lean, the proof assistant used here, checks a proof of a precisely stated proposition. A separate record links selected propositions to the wording approved for the paper. Release checks then test those recorded relationships when files change. A change to a proof's hypotheses can therefore trigger a review of the explanation that depends on it, rather than leave the paper describing an older statement.

Several parts have close precedents. Agent Hunt coordinates formalisation work, LeanMarathon maintains a shared proof blueprint across long agent runs, and Lean Atlas narrows the declarations that require semantic inspection [20, 1, 6]. Our contribution is architectural composition around a continuing problem record. Tao's account of proof generation, verification, exposition, and digestion motivates including the explanatory work in that record [13]. The comparison in Section 11 identifies the overlaps.

Authorship and AI use. Will Cook built and directed the research infrastructure. AI agents produced the mathematical interpretations, proofs, prose, and software. Cook maintains the project and is responsible for its presentation; this is not a claim of personal mathematical verification or independent specialist review.

1.1 What the system records

We use *problem world* for the collection associated with one open problem: its statement, literature, partial results, experiments, and failed approaches. A query selects a bounded neighbourhood inside a problem-sized world. For example, an attempt to weaken a theorem's hypothesis needs that theorem, the declarations its proof uses, and known counterexamples to the weaker claim. It rarely needs the entire library.

The stages require different checks. Concurrent agents can overwrite each other's files. A proof can establish a proposition that was stated incorrectly. A paper can exaggerate a correctly proved result. Table 1 shows how the implementation handles these different failures.

Operation	Record	Check
Select a question	Relevant statements and source references	Inspect the selected context and its omissions.
Edit shared files	Time-limited claim on named paths	Reject overlapping work claims.
Run a proof check	Source revision and build result	Use the pinned Lean environment and available build slot.
Describe a result	Statement, evidence, and hypotheses	Reconcile the formal and informal claims.
Publish a revision	Reviewed wording and source links	Check the registered publication relationships.
Request external review	Selected results and review outcome	Record the review separately from local acceptance.

Table 1: Checks at different stages of the research process.

The system also records repairs to its own tools. A missing source link may lead to a better query; a repeated build conflict may lead to a scheduling change. These are changes to the research process. Mathematical records change when a proof, counterexample, computation, or reviewed statement supplies the corresponding evidence.

2 The whole lifecycle in one picture

Figure 1 follows an attempt from question selection to publication. A failed proof returns a specific unfinished obligation to the researcher. A reviewer may instead find a misstatement or an overlooked source. The next attempt starts with that correction recorded.

The agent selects a question and gathers the statements and evidence needed for it. It then runs an experiment, edits a proof, or writes an explanation. The result and its checks are saved with the source version, so another attempt can use them. A finding may also suggest a reusable lemma, a better query, or a repair to the tools.

The public proof-cockpit command summarises the checkout, Lean toolchain, open propositions, and saved research sessions. Its check mode runs the claim-registry, cold-clone, and projection-freshness checks. These establish whether the repository records agree with their sources; checking a proof requires a separate Lean run. The commands are listed in Appendix A.

2.1 Instructions for recurring jobs

The public clone provides a skill file for each recurring job. It tells an agent where to start, what it may change, what evidence to save, and when to stop or hand the work on. Figure 2 shows the ordinary research path.

The public skills are instructions an agent can execute in a clone. They cover orientation, research, proof checks, updating affected papers and records, and preparing a pull request. The reproducibility appendix and the agent entry link to the individual commands.

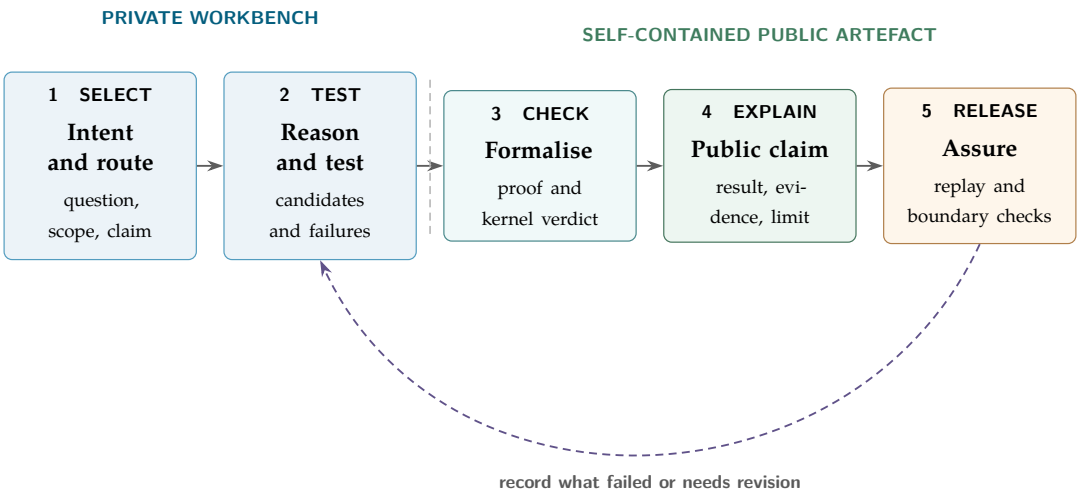

Figure 1: The end-to-end architecture. The dashed vertical line marks the release boundary. Everything to its right remains usable without the private workbench.

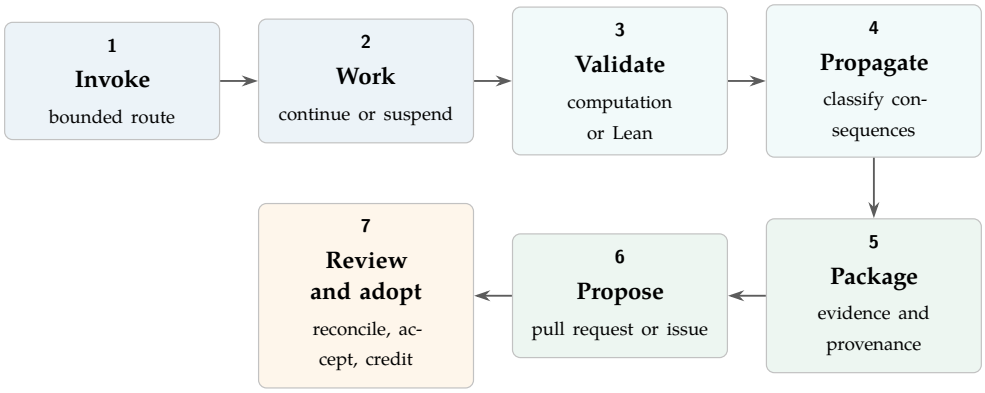

Figure 2: A contribution from the initial task to its inclusion in the repository.

3 The private workbench

3.1 Disk is shared memory

Durable state lives in files. The workbench stores the task, source changes, check results, and unfinished work so another agent can resume without relying on the previous conversation. A *receipt* is a saved record of an operation and its outcome; a *projection* is a generated view of the underlying records, such as an index or status page.

A router selects a short set of relevant records and instructions for the current task. Each summary points to its source, which the agent can open when it needs more detail. The mathematical query mechanism is described in Section 4.

3.2 Type A and Type B

Type A is an agent working in a harness such as Claude Code or Codex. It can read the checkout, run tools, edit files, and test the result. Type B is a one-shot LLM exchange outside that harness, such as Astra Pro in the web app. The operator supplies a research packet and brings the response back into the checkout, where the harness agent checks and integrates it. The labels describe substrate access, not model quality.

The public `source_packet.py` tool exports selected committed files into one readable attachment. It records the source commit and file hashes, preserves the complete selected text, and offers an offline checksum check. An operator can therefore send the same statement and surrounding evidence to a web-app model without asking it to navigate GitHub. The operator chooses the sources; the tool does not infer their dependency closure. Binary files are preserved as labelled base64 and require decoding before inspection.

The private packet builder also prepares for two ways of reading. A browser-only recipient receives links to the public frontier and its source statements. An offline dossier carries the selected proofs and their dependencies. When it includes a runnable checker, the builder also packages its declared local runtime dependencies and rejects missing ones. This lets a recipient with code execution test the supplied calculation without reconstructing the author's checkout. The returned answer still goes through local checking and integration.

3.3 Concurrency without shared-state confusion

Several agents may work at once, but concurrency is admitted only where the write scopes can be separated. A work item identifies the objective and expected evidence. Before mutation, an agent claims exact paths for a bounded lease. Independent claims may proceed concurrently; overlapping claims must be coordinated or deferred. Fan-out is followed by a fan-in barrier where the results are compared, validated, and integrated.

Coordination state is written to append-only ledgers and immutable receipts; generated status pages project those records. Bounded job counts and focused Lean builds keep concurrent work within the machine's capacity.

3.4 Routing and scheduling

The router exposes a typed option surface: searchable entries for tools, instructions, source files, and ongoing work. An agent first receives a short card, then follows its links to the working context or source. The selected entries remain visible, so a missing premise or poor retrieval choice can be investigated. The initial response gives the next action and a route to the full record. Detailed audits stay available without being repeated at every entry; task-specific constraints remain in the selected context.

For a mathematical query, the compiler assembles the target statement and premises for possible arguments. Feedback revises the affected argument while retaining earlier counterevidence and diagnostics. The packet records source fingerprints; a freshness check compares them with the current inputs and requests recompilation if they differ.

The work ledger separates past work from present permission to edit. An agent claims exact paths for a bounded lease. A directory claim conflicts with claims on its children, and a lease expires even if its holder crashes. Completion, reopening, and replacement are recorded as separate events. The status view reads these records; it does not issue new permissions.

A maintenance daemon converts repository events into jobs. It uses a SQLite store in write-ahead-log mode for the events, jobs, run results, and resource claims. Hooks, file scans, interruptions, and explicit commands feed the store. Content hashes remove duplicate events; an allowlist restricts the jobs that may run; a single-daemon guard prevents competing schedulers. During one recorded snapshot, the status page reported the daemon as not running while queued jobs remained visible. The queue persisted, but the service was inactive.

3.5 What continuous mathematical work looks like

A continuous goal resumes from the latest accepted research state: sources read, routes eliminated, computations interpreted, and formal obligations still open. Each run selects a useful next step, such as a lemma, a counterexample, or a sharper reduction, and records what changed.

A representative internal trace makes this less abstract. Over fifteen successive turns, one problem-directed run recorded 313 visible progress updates and 3,491 command events, with linked workers exploring separate analytic, computational, and formal lanes. The movement was not a straight line from prompt to proof. Numerical probes exposed structure; exact arithmetic replaced promising samples; several attractive strengthenings were killed by counterexamples; partial inequalities were narrowed to

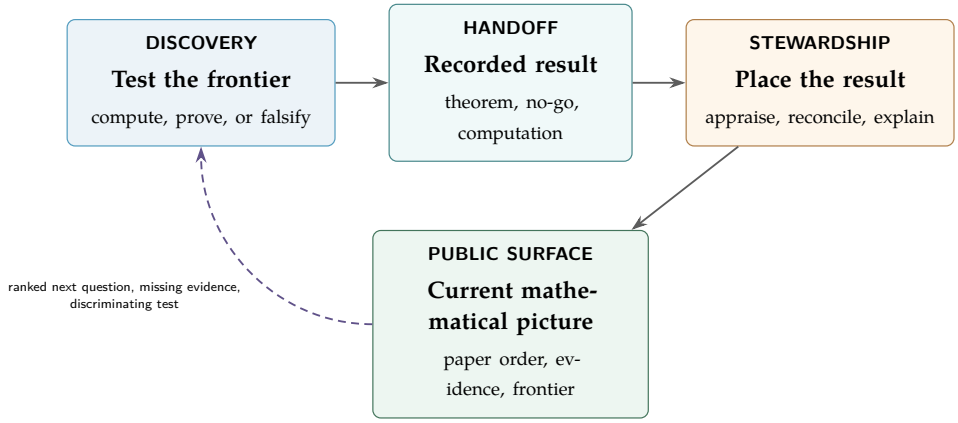

Figure 3: Search produces a result; a second pass checks its place in the existing work and updates the account used to select the next question.

their valid domains; Lean targets were attempted only after an ordinary mathematical kernel had stabilised; and successful local results were propagated into the problem frontier before the next search began. While long exact computations ran, the controller advanced independent work instead of repeatedly polling them.

That trace also displays why the architecture keeps several evidence layers. Its compact narrative reported no observed changed paths even though its own turn summaries referred to landed commits. This is not evidence that nothing changed, nor that the summaries were correct. It is evidence that a compressed trace has an observation boundary. Repository state, command results, diffs, formal builds, and commit objects must settle the question. Likewise, four turns lacked complete closeout events and only sixteen of forty-two linked session outcomes appeared in the compact projection. Completeness is therefore an explicit field, not an impression created by fluent prose.

Each advance leaves an authority-bearing artefact and receipt. When the next step needs an unavailable validator, a human decision, or a new idea, the run preserves its unfinished obligation and a useful re-entry point.

3.6 Coupled continuous goals: discovery and stewardship

The public workflow separates proof search from maintaining the account of what has been found. A discovery task investigates one mathematical question. A stewardship task compares the result with the existing literature and formal development, decides where it belongs, and updates the affected paper and records. One agent may do these jobs in succession, or separate agents may handle them. The separation makes time for questions that proof search alone can neglect: whether the result is already known, whether several declarations express one theorem, and whether the paper explains the hard step.

A stewardship pass can change the next research question. An apparently new result may be a routine corollary; an overlooked theorem may already supply a missing premise. The pass records the evidence, judges mathematical interest, revises the explanation, and recommends further work. Proof status and editorial prominence are recorded separately.

The `run-coupled-research-goals` skill invokes the search and update jobs with a shared source revision. New results or corrections can trigger another pass. If the repository has not changed, the jobs yield. Contributors who need an introduction can first ask `explain-public-system` to read the papers and explain one result at the requested level.

4 The mathematical reasoning loop

Research begins with the mathematical question and the relevant literature. The agent compares possible approaches, tests the promising ones, and formalises steps once their statements are stable. The choice

of approach is an authored judgement: a counterexample that rules out an expensive line of work may deserve more attention than many easily proved lemmas.

4.1 Experiments are route selectors

Computation serves three useful roles. It can find a counterexample, measure a finite pattern, or test whether a proposed mechanism is plausible enough to formalise. Each experiment records its inputs, code, finite domain, output, and interpretation. The interpretation must state what the experiment cannot show.

A finite computation becomes formal evidence only through an explicit bridge. For example, Lean may evaluate a finite proposition with `decide` and check the resulting proof term. Even then, the conclusion remains finite. A pattern observed for many inputs does not become an "all inputs" theorem, and a successful numerical approximation does not become an exact equality. Failed experiments are kept when they prune a natural route; otherwise later agents pay to repeat the same mistake.

Failure modes are mathematical outputs. The system distinguishes three kinds of negative evidence. A failed agent attempt says only that one search path did not close. A counterexample can refute a conjecture on its exact domain. A Lean no-go theorem can rule out a whole strategy class under explicit hypotheses. The last two may be as valuable as a positive lemma: they prevent repeated work and reveal which new ingredient an endpoint requires. This corpus therefore keeps deep problem-specific reasoning surfaces, proof gaps, coefficient-only and fixed-precision obstructions, and corrected statements beside successful theorems. Every no-go keeps its scope visible; failure of one mechanism is not failure of every possible proof.

4.2 Following a change of hypothesis

The composite-dilation work gives a concrete example. A support here selects the exponents in a Mersenne subseries. A researcher wants to extend a multiplication identity from supports consisting of primes to more general supports. The public query tool retrieves the existing statement, its source module, and the claim that uses it. The researcher can then inspect which use of primality must be replaced, rather than beginning with the whole library. The relevant formal source counts the extra support divisors introduced by multiplication as an explicit defect term, and proves that this term vanishes on prime support. The extension has therefore become a precise question about that term.

During an attempt, the agent uses a session record to keep observations, conjectures, plans, and interpretations together. When it asks Lean to test a candidate, the proof workbench saves the submitted source, computes its digest, runs the probe, and appends the result. A later success record must cite an accepted probe whose stored source still matches that digest. The agent can revise its next candidate while leaving the earlier attempt available for inspection. A reviewer follows the resulting argument and checks that the explanation describes the statement actually tested. The public proof workbench implements this record-and-check step.

The useful memory of the failed extension is more specific than "composite support did not work". Its failure record names the lost hypothesis, the two supporting declarations, and the repair: retain the defect and prove it vanishes or is controlled on the chosen support. It also names the surviving mechanism and the condition for reopening the route. The theory-laboratory builder joins that record to the declaration and semantic graphs; its checker verifies that the cited objects and neighbouring mechanisms exist. The next agent receives both a reason to abandon the unmodified identity and a concrete direction to study.

An accepted change can leave several files out of date. The declaration index reads the Lean source; source-coordinate records join that index to the papers; the corpus descriptor summarises the resulting state; and the publication entry packet reads the final claim and publication records. The projection refresher orders these builders by their dependencies. Release checks compare the stored projections with what their owners would regenerate. These updates keep the paper, query results, and formal source consistent for the next contributor.

4.3 Checking the result

Three checks answer different questions. The literature establishes the reported status of the problem. Lean checks the exact formal statement and proof. Mathematical review compares that statement with the intended question and with prior work. The record preserves the source and result of each check.

When an argument comes from the literature, the citation remains attached to the formalisation. When an experiment or failed proof is retained, its scope and interpretation remain attached too. This lets later researchers examine why an approach was pursued and what the attempt established.

4.4 From local progress to reusable mathematics

A lemma proved for one Erdős problem may be useful elsewhere. Preparing it for a shared library requires another pass: determine which hypotheses the proof uses, search for an existing theorem, and state the result in a form that other developments can use. The general proof must be checked, and the original result should follow as a clear special case.

The repository can preserve that starting point and help prepare a submission. Mathlib's contributors and reviewers decide whether it belongs in the library [16]. Credit should distinguish the original observation from the generalisation, formalisation, and library work. The strategy paper describes this proposed route.

4.5 Problem-sized Lean worlds and bounded theorem neighbourhoods

The declaration atlas locates formal statements. Dependency maps show which declarations a proof uses, while authored relations explain their mathematical role. The claim record links selected results to their public descriptions. A researcher uses these views together when deciding which source to inspect.

The scale is already problem-sized. At the August 2026 snapshot the attached public corpus described 1,024 Lean modules and 153,396 declarations. The much deeper private record for Problem 1041 contained 177 exact results, seven open producers, 72 negative results, 134 Lean modules, and 335 executable experiment programs. Its public research-corpus manifest alone recorded 685 files and 35 selected strongest results. These are dated inventory facts, not a throughput benchmark; generated certificate families also make raw theorem counts a poor measure of mathematical value.

At that snapshot, the canonical public 1041 library had two integrated Lean modules, while its separately governed public research export has 125 Lean sources and its private world is larger again. The public route memory still records no reviewed 1041 claim family. Across the whole semantic corpus, structural linkage is broad, but only 25 statement nodes and eight relations carry digest-bound semantic-review receipts at this snapshot. These differences are intentional status boundaries: a discoverable declaration is not thereby understood, integrated, reviewed, or publishable.

An agent does not receive this world as one prompt. First the problem cockpit fixes the endpoint, status, claim ceiling, and canonical frontier. It keeps open producers distinct from the target so a promising route cannot silently replace the problem. Next a bounded neighbourhood selects the strongest relevant results, exact source coordinates, imports, consumers, alternative formulations, experiments, counterexamples, no-gos, and literature. It labels every item by evidence class and emits an omission receipt with an expansion route. One 1041 packet exposed only four of 134 Lean modules, four of 335 experiments, and four of 55 no-go notes; designed omission, not exhaustive loading, made the context usable. When an exact path is claimed, a local connection card can put its prerequisites, sibling mechanisms, consumers, falsifiers, and validation target before broader retrieval.

4.6 Comprehension before the mathematics

The private working-memory compiler separates orientation from focused work. An overview shows the available results and open questions so that an agent can choose a target. It does not construct candidate proof routes. A focused request names a claim, declaration, or premise; if no target can be resolved, it is reported as unanchored and returns no proof branches.

Each attached corpus retains its own index; nothing is copied into a central index. Compact descriptors and content fingerprints identify the sources. The compiler opens a larger atlas when the bounded query does not supply enough context. The focused packet starts with the target, established results,

and missing premises. Its workspace groups premises into candidate arguments and records tests or counterexamples that could reject them.

A freshness check compares the recorded projection and descriptor fingerprints with their current values and requests recompilation when they differ. An exactly named Lean definition can also be newer than its index. In that case, the compiler returns the current definition and use-site coordinates, and defers broader graph-based routes until the index is refreshed.

When the selected material exceeds the context budget, the compiler first shortens summaries and repeated branches while preserving source identities and retrieval routes for omitted details. Only if the resulting packet still does not fit does it report the requested and estimated size and exit non-zero. These checks make the selected context inspectable. Whether it leads to better mathematics requires the comparison proposed in Section 11.

4.7 Consequence propagation

After a declaration changes, the agent checks the files that may depend on it: imports, later proofs, experiments, claim records, and papers. The dependency map identifies candidates; a semantic second pass determines which actually need revision. Each is updated, checked as unchanged, deferred with a reason, or excluded from the change. No projection may bulk-strengthen a family of claims.

The public `propagate-research-consequences` skill gives instructions for this pass. Work from an older clone is checked against its original revision and again after integration with current main. The original contribution and the integration changes retain separate attribution.

A recurring process failure may also justify changing a tool or instruction. The proposed repair records the local failure, other affected cases, and a check of the repair. The implementation supports this form of feedback; it has not measured how often useful lessons are successfully retained.

5 The public Lean repository

The public checkout begins at a deliberate boundary. It contains every file needed to inspect its claims and replay its checks. It does not call back into the private workbench, and no public theorem depends on an unpublished private lemma. The larger workflow is provenance: it explains production, not truth.

The repository has two Lean roots. The reviewed root contains the established 249/257 publication lane. The problem-owned expansion root contains work on six additional Erdős problems and unpromoted lanes for 249 and 257. Both roots are checked by the same Lean kernel, but kernel acceptance does not automatically promote a declaration into a reviewed public claim. Promotion is a separate change to the claim record and exposition.

The main public sources have deliberately separate jobs:

Surface	Authority	It does not establish
Lean source and pinned toolchain	Exact formal statements and proofs accepted by the kernel.	Intended meaning, novelty, significance, or faithful prose.
Reviewed claim record	Approved wording, status, evidence, bounded domain, and adjacent open statement for selected claims.	Correctness or completeness of the human review.
Papers and guides	Human explanation and reading order within the recorded claim ceiling.	New formal authority.
Generated maps and query tools	Bounded navigation across declarations, problems, graphs, papers, and claims.	Proof or permission to strengthen a claim.
Release checks and continuous integration	That configured identities, relationships, generated files, licences, and negative tests pass on a named revision.	Understanding of unrestricted prose or independent mathematical approval.

Table 2: The public authority split.

The companion public Plectis repository has a different role. It publishes runnable, bounded mechanism slices and receipts from the wider system. The Lean repository publishes a mathematical corpus. Neither public repository inherits authority from the other, and neither is a public mirror of the private root.

6 Comparator, Palomar, and publication

6.1 Comparator: an exact-statement firewall

Comparator protects a narrow but important boundary. Selected propositions are declared again in a challenge module without their proofs. A solution module must provide terms of those exact types. The configuration pins the permitted axioms, the modules, and a runtime receipt. A named altered statement must fail, which checks that the harness has not become vacuous.

Comparator therefore answers: "Does the proof-bearing corpus still implement this separately stated Lean interface under this axiom budget?" It does not answer whether the interface is the right translation of an informal problem, whether the theorem is new or important, or whether an independent human has reviewed the proof. "Comparator-checked" is accurate; "independently verified" is not.

6.2 Palomar: selecting what deserves review

Comparator's roster supplies evidence for a review portfolio. To prepare that portfolio, maintainers compare result families in a local selection record. Each assessment explains the consequence, hard mechanism, evidence, and open remainder, then records why the family is selected, represented by another result, deferred, or set aside. The local Palomar qualification checker checks that this record covers its declared candidates and that the selected packet satisfies the submission requirements. The mathematical ranking is authored; the checker validates its supporting record.

This separation matters under proof abundance. Counting theorems rewards generated volume and routine closure. Selection instead asks which result changes the mathematical picture, which conditional route is closest to an endpoint, which obstruction prevents wasted work, and which explanation will help an expert assess the claim. The ranking may guide attention; it cannot alter proof status.

6.3 Publication and propagation

After formal proof and statement reconciliation, a result may enter an authored paper, a claim record, a Comparator packet, and a Palomar review unit. Each is a separate projection with a separate ceiling. A public release is made only after the Lean build and the release-surface checks pass, and publication remains a human action.

The return path is equally important. A proof failure may improve a formalisation heuristic. A counterexample may close a family of tempting routes. A reviewer objection may require a narrower public claim. A release drift may create a stronger check. Those lessons propagate to the smallest durable owner—a theorem, experiment record, skill, standard, route, or claim boundary. They do not rewrite raw intent, generated views, or mathematical status by implication.

This places the repository inside a longer mathematical lifecycle: candidate, proof, exact checking, exposition, independent assurance, publication, expert digestion, and eventual community canonicalisation [13]. The system directly supports the early and middle stages. It can prepare later stages but cannot award them to itself.

6.4 What the papers should explain

Tao's essay follows proof generation, verification, exposition, publication and community digestion through to the adoption of useful mathematics [13]. His discussion of exposition is especially relevant to a repository largely written by agents. Fluent prose can spend pages on routine steps while hiding the step that took real work. It can also erase the *natural friction* that tells a reader where to slow down.

The problem papers should therefore explain why an approach was chosen, where it becomes difficult, and what an unsuccessful attempt teaches. The failure records supply evidence for that explanation: a lost hypothesis, an exact counterexample, or a proved obstruction. Tao's later discussions make the case for preserving such insight while a problem is still open [14, 15].

Writing can reveal errors in the research record. An unexplained implication may conceal a missing premise; a confusing paragraph may combine distinct results. The author must return to the statements and repair the account.

The writing skill also asks the agent to compare the manuscript's selection with the actual source tree. A useful theorem can be missing from the paper or from the query that supplied its outline. The agent reads omitted results and their uses before deciding whether the account should change. When several proved ingredients appear to settle a step still described as open, it writes out their composition and checks the assumptions at each connection. If that argument needs further formalisation, the missing step becomes a proof task. The paper can explain the ordinary argument while identifying precisely which part has passed Lean.

A corrected account must reach the reader. The agent updates the existing result records and reading order, rebuilds the affected views, and inspects the actual query output. A registry edit alone can leave a result absent from the front door. During concurrent work, publication views are generated from a committed snapshot so that another agent's unfinished proof cannot enter the published evidence accidentally.

This paper was itself produced within the system. Its evaluation is author-operated.

7 One complete boundary: finite is not unbounded

Erdős Problem 249 asks whether

$$S = \sum_{n \geq 1} \frac{\varphi(n)}{2^n}$$

is irrational, where $\varphi(n)$ is Euler's totient function [3]. The development reduces irrationality to a family of exact non-integrality certificates. Along the diagonal $H_t = \operatorname{lcm}(1, \ldots, t)$, write $\mathrm{Cert}(t)$ for the existence of the required finite certificate.

Lean checks the finite theorem

$$\forall t \leq 82, \quad \mathrm{Cert}(t).$$

The endpoint needs an unbounded supply:

$$\forall T, \quad \exists t > T, \quad \mathrm{Cert}(t).$$

The development proves that the unbounded statement is equivalent to the irrationality claim. It does not prove the missing implication from the finite range to the unbounded statement. No matter how large a fixed checked bound is, a larger cutoff exists.

This example passes through every layer. Computation constructs finite data. Lean checks the certificate predicate and the finite theorem. The formal graph records its dependencies. The semantic graph identifies its role. The claim record states the finite range and names the unbounded requirement as open. The paper explains why the quantifiers differ. Comparator checks selected exact interfaces. Palomar may rank the result family for review. The release checker requires the limitation to remain visible.

The last check was added because the boundary once escaped. A historical README edit changed a clause saying that the finite cases did *not* supply the open requirement into one saying that they completed it. Lean was untouched, and the release checker passed because that prose relationship had not been registered. After the relationship was added to the claim record, a deliberately false copy was rejected. This is evidence for one failure and repair. It is not a detection rate, a proof that every claim-bearing sentence is registered, or an evaluation of mathematical review quality.

The historical study was narrower than a benchmark. In that study, nine of the ten edits were rejected. One escaped because the relationship had not been registered. The original run logs were not retained. The edits were authored by the checker's author. After the relationship was registered, only the escaped edit was reconstructed against the repaired checklist. The other nine edits were not rerun against the extended checklist. The post-repair witness accepts the current README and rejects a test copy containing the false clause.

The same architecture handles other shapes of progress. Problem 257 separates a theorem for full-support representations from a stronger arbitrary-support statement that remains open. Problem 68 has an exact carry-based equivalence but no theorem producing the required carries. Problem 251 has an exact series reformulation rather than an irrationality proof, and Problem 243 records a conditional recovery theorem whose premises remain part of the public claim. A counterexample or corrected statement, as in the 1041 lane, is also a first-class result. The publication rule is identical in every case: preserve the quantifiers, premises, and nearest stronger open statement.

8 Inspection routes

A reader can inspect the public system through short, question-shaped routes. The labels below link to repository paths; the paper does not print the full URLs.

Question	Start here
How does the public repository fit together?	`ARCHITECTURE.md`
What may the project say, and what remains open?	`docs/claims.json` and `docs/methodology.json`
Where is the checked mathematics?	`Erdos249257.lean` and `ErdosProblems.lean`
How can I navigate without reading the whole corpus?	`docs/ORIENTATION.md` and `scripts/query_corpus.py`
What exactly does Comparator check?	`docs/EXTERNAL_VERIFICATION.md` and `verification/comparator.json`
What does Palomar select and qualify?	`docs/PALOMAR_QUALIFICATION.md` and `docs/PALOMAR_RESULT_SHOWCASE.json`
Which papers exist and what question does each answer?	`docs/papers/README.md`
Which checks gate a release?	`scripts/check_release.py` and `.github/workflows/lean.yml`
How can work return with public credit?	`CONTRIBUTING.md` and `research-commons/CONTRIBUTIONS.md`

Table 3: Public files for inspecting the source, checks, and contribution record.

9 What can be trusted

The architecture supports a chain of narrow conclusions:

1. a recorded experiment ran on its stated finite inputs;
2. a pinned Lean kernel accepted its stated formal source;
3. a maintainer approved a selected interpretation and public boundary;
4. Comparator matched selected proof-bearing declarations to separately stated interfaces and an axiom budget;
5. Palomar's local checks validated the structure of a selected review packet;
6. the release program found every configured public relationship intact on the named revision.

The checks have a common limitation: agreement between files is weaker than independent examination of their content. A coordinated error in source, claim record, and prose can pass a structural comparison. Unregistered prose can overclaim, and literature status can change. The system does not require a second independent mathematician to approve every result.

This is also a limitation of assurance cases: recording a relation between evidence and a claim does not establish that the evidence is sufficient [12]. The links and negative tests make selected claims easier to inspect, while mathematical judgement remains necessary.

10 Scaling from one clone to a search network

10.1 Clone, attach compute, and mine bounded results

A contributor can use a different agent runner with the public source and checks. Several workers can investigate separate statements or experiments, then return their results for integration. The repository

calls this *result mining*. Each return names the result, evidence, starting revision, and unresolved question.

As the number of workers grows, someone must still compare their returns with the literature and existing corpus, update the papers, and choose the next questions. This is the stewardship work described earlier. A large run may produce no new result; a short counterexample may change the research plan. The number of workers does not determine the amount of useful work to publish.

The controller limits concurrent work according to file conflicts and machine capacity. Reading and small calculations can proceed while another task holds the Lean validation slot.

Lean validation uses semantic single-flight coordination. A request is identified by the reachable source, logical targets, toolchain, dependency lock, and authority mode. Its command string and working directory do not determine request identity. Equivalent requests share one owner and may reuse a completed receipt; changed inputs require a fresh build. Distinct requests that would still compete for the same host-wide Mathlib resource are serialized by a shared conflict key. A distinct request that cannot acquire the slot ends with a resource-busy deferral before its validator starts. The agent can continue independent work, then resubmit that request after the resource is free. Waiting on the deferral receipt does not start a build. An equivalent request already running can be joined instead.

These are four separate scaling limits: the number of research attempts, safe concurrent editing, proof-check throughput, and expert review. Increasing one can leave another as the bottleneck.

Returning work and recording credit. A contributor can fork or clone the repository and submit a pull request or research-progress issue. The return records the result, starting commit, evidence, collaborators, and tools used. A maintainer supplies an explicit review decision and the commit containing the accepted work. The acceptance program checks the submission and commit ancestry, then copies the return into an accepted receipt while preserving its result, evidence, and attribution. Only an accepted receipt enters the generated contribution views. Their builder reads receipts committed in the checked-out history and rejects a receipt whose working copy differs from that committed source. The published credit therefore identifies the accepted work and the review decision that admitted it. Acceptance, mathematical claim status, and release inclusion remain separate decisions. Later corrections preserve the earlier record and its attribution.

The starting commit lets a maintainer reproduce the original contribution before adapting it to current main. A substantive conflict resolution receives separate integration credit. The contribution protocol and credit policy give the details. The cold-clone study examines the navigation used to begin this work.

The public clone supplies the corpus and checks, but the private orchestration system is not a turnkey public service. Large multi-provider deployments remain a design target rather than a reported benchmark. No outside contributor had completed this path by 31 August 2026.

10.2 Using records of unsuccessful approaches

A recorded obstruction should identify the method and hypotheses it excludes, along with variants that remain possible. Linking these records to theorems and experiments may help an agent avoid repeating a failed approach or spot a related obstruction in another problem.

Graph-conditioned models could be tested on these records. For example, training data might pair a tempting argument with a counterexample or theorem that explains its failure. These are proposals for future systems. A mistaken or over-broad relation could instead hide a viable method, so an evaluation would need reviewed source records and held-out tasks.

10.3 An experimental agenda

The public benchmark builder prepares source snapshots from before a target declaration was introduced. It checks that the declaration is absent and filters added graph and mechanism records against the declarations available at that revision. The snapshot omits Git history, since a linked worktree would let an agent recover the later proof from the shared object store. The answer key and introducing-commit metadata remain outside the packet. These are prepared inputs for an evaluation; the runner must still isolate filesystem and network access, and the selected material needs review for answer hints.

The next evaluation should ask more than how many theorems additional compute produces. It should compare graph-aware and graph-blind agents on frontier selection, repeated-dead-end rate, time

to a useful obstruction, premise discovery, proof completion, and calibration of public claims. It should test transfer between unrelated mathematical domains as well as between neighbouring Erdős problems. Independent teams should replay result packets and review whether no-go edges are scoped correctly. The central scaling question is whether an increasingly rich map of what works and what cannot work makes future reasoning more efficient and more original, or merely makes the system more confident about the territory it already knows.

11 Relation to other approaches

Theorem-proving agents. This architecture is not a new proof-search algorithm. LeanDojo and Pantograph provide programmatic Lean environments and proof-state interaction [17, 18]. OpenProver already combines a planner, parallel workers, independent verifiers, compact working memory, a larger repository, and Lean feedback [19]. Agent Hunt already studies concurrent formalisation with locks, bounties, guarded ownership, and collaborative agents [20]. LeanMarathon uses an evolving Lean blueprint as both proof graph and shared record, with contract-scoped agents, adversarial target review, and parallel CI-gated proof work [1]. It also stores explanatory prose beside Lean types and checks agreement between cited and elaborated proof dependencies. This overlaps both the durability and checked-exposition concerns here. DreamProver learns a compact reusable lemma library through wake–sleep cycles [21]. These are baselines for proof interaction, parallel search, and learned proof memory. The present work follows the result into the public research record, including its explanation, open obligations, and contribution history.

Graphs and checked exposition. Proof blueprints connect informal proof plans to named Lean declarations. `leanblueprint` checks that author-supplied declaration names exist, while LeanArchitect infers formal dependencies and unfinished-proof status and exports synchronised blueprint material [4, 5]. The graph layers here have a related navigational role, but the public-claim record begins after a result has been selected and asks which wording and open boundary were reviewed.

Semantic-audit systems address a neighbouring problem. Lean Atlas narrows the declarations a person must inspect for chosen theorem statements, conditional on the semantic correctness of the returned set and trusted base [6]. EconCSLib uses models to translate and compare formal and informal statements, with saved human judgements for a subset [7]. The present architecture uses models throughout production but assigns none of them final semantic authority. Their output becomes evidence or a candidate until a source-specific gate accepts it.

Checked-document and traceability systems make heterogeneous relationships explicit. Isabelle/DOF places formal and informal material in a typed checked document [8]; requirements traceability follows commitments across development and revision [9]. This repository keeps prose unrestricted and records selected public boundaries separately. That choice makes adoption simple, but leaves unregistered prose outside the checker.

Auditable scientific agents. HEP makes hypotheses, evidence, belief updates, lineage, and resolution states explicit in an append-only registry [22]. Symposium proposes immutable community publication histories with attributable artefacts, declared evidence, assumptions, and purpose-sensitive arguments [23]. EurekAgent treats permissions, artefacts, budgets, and human supervision as first-class parts of an agent environment [24]. Persistent records and environment design are therefore not unique to this system. The narrower distinction here is between authorities that must not be collapsed: experimental evidence, kernel acceptance, reviewed interpretation, exact-statement assurance, editorial selection, and public release.

Mutation testing asks whether a test distinguishes a seeded fault from the original program [10, 11]. The deliberately false boundary in the worked example follows that idea. Because the examples were selected by the system's author and were not run as a controlled external evaluation, they show that particular checks can fail; they do not measure overall adequacy.

Contribution and limits. The contribution is the implemented connection between ongoing research and its public mathematical record: formal statements, interpreted results, scoped failures, contribution credit, and checked publication relationships. LeanMarathon already addresses durable agent coordination and statement fidelity; HEP already records hypotheses and refutations. These overlaps make

component-level comparison more informative than a claim to cover the most stages. The evidence reported here is a design and worked reconstruction from one evolving corpus, not a controlled comparison of research productivity.

Scaling beyond this corpus. The design is not tied to Erdős problems. A new domain needs stable result identities, source and status records, a formal checker where one exists, and an explicit public-claim boundary; it need not use the same mathematics or even Lean. Larger deployments should federate independently owned corpora rather than build one authority database, separate producer and reviewer teams, benchmark each boundary independently, and preserve immutable result generations. Proof-search layers can adopt distributed task markets, persistent lemma learning, and richer human steering, while publication layers can add external replay, signed review receipts, and cross-project claim links. None of these extensions should allow learned process memory or a graph edge to change mathematical truth by itself.

12 Conclusion

Plectis connects an ongoing research process to a public record that another contributor can continue. The working example shows the intended unit of reuse: a failed extension leaves a correction term, its formal statement, and a precise next question. The surrounding machinery records attempts, checks statements, and refreshes the linked explanations and source views.

The present evidence is a working implementation and author-operated cases. The next evaluation is whether outside contributors can recover the relevant mathematics, return reproducible work, and reduce the effort required for the following attempt. That test should measure understanding and reuse as well as proof completion.

A Reproducibility

Public first contact. A fresh public clone can reproduce the control card and structural checks:

```
python3 scripts/proof_cockpit.py --format card
python3 scripts/proof_cockpit.py --check
```

The first reads committed public metadata; the second checks the claim registry, cold-clone contract, and orientation freshness. Neither checks a proof. Formal authority begins with the pinned Lean build named by the card.

Dated navigation counts. At the semantic review's snapshot, taken before the eight-problem consolidation of 2026-08-02 enlarged the corpus to 153,238 declarations, all 151,761 then-live declarations were inventoried and routed, and all 144,487 author-written theorem-like declarations had an exact node link. Of those, 139,773 (96.7%) participated in authored mathematical interpretations: 3,285 as exact proposition evidence and 136,488 as bounded certificate- or module-family context. The remaining 4,714 were linked only through exact source-module and normalised-signature families, not authored mathematical paraphrases. Every declaration selected for a public claim had an authored route. The command `python3 scripts/query_semantic.py coverage` derives these volatile navigation counts and checks their references. They do not measure semantic review quality or public-claim completeness.

The paper inventory, `docs/publication_contract.json`, records source and PDF cryptographic hashes and validation commands. The worked-example evidence is recorded in `docs/publication_evidence.json`. The reconstruction file `experiments/publication_mutations.json` specifies the deliberately false edits and their limitations. Comparator's public configuration and receipt are named by `verification/comparator.json`; Palomar's local readiness and ranking surfaces are named by the corresponding files under `docs/`. These artefacts identify what was checked. They do not interpret unrestricted prose or confer external acceptance.

The private system is described here at the architectural level because it is production provenance, not a dependency of the public result. Private paths, operator material, unreleased work, and private ledgers are neither required nor granted authority by this paper. The public checkout remains the inspection and replay boundary.

References

[1] Y. Zhang, Y. Sun, T. Suzuki, J. D. Lee, and F. Liu, *LeanMarathon: Toward Reliable AI Co-Mathematicians through Long-Horizon Lean Autoformalization*, 2026, arXiv:2606.05400, Sections 2–4.

[2] L. de Moura and S. Ullrich, *The Lean 4 Theorem Prover and Programming Language*, in *Automated Deduction—CADE 28*, Lecture Notes in Computer Science 12699, 2021, pp. 625–635, DOI.

[3] P. Erdős and R. L. Graham, *Old and New Problems and Results in Combinatorial Number Theory*, Monographies de L'Enseignement Mathématique 28, 1980, p. 61.

[4] P. Massot, *leanblueprint*, plasTeX plugin for Lean formalisation blueprints, 2020, software repository.

[5] T. Zhu, P. Monticone, S. Welleck, and J. Avigad, *LeanArchitect: Automating Blueprint Generation for Humans and AI*, in *17th International Conference on Interactive Theorem Proving*, LIPIcs 382, 2026, pp. 25:1–25:16.

[6] B. Yanahama and A. Sannai, *Lean Atlas: An Integrated Proof Environment for Scalable Human–AI Collaborative Formalization*, 2026, arXiv.

[7] N. Garg, *EconCSLib: AI-Assisted Lean Formalization for Economics & Computation Research*, 2026, arXiv.

[8] A. D. Brucker and B. Wolff, *Isabelle/DOF: Design and Implementation*, in *Software Engineering and Formal Methods*, 2019, pp. 275–293.

[9] O. C. Z. Gotel and A. C. W. Finkelstein, *An analysis of the requirements traceability problem*, Proc. First IEEE International Conference on Requirements Engineering, 1994, pp. 94–101.

[10] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, *Hints on test data selection*, IEEE Computer 11(4), 1978, pp. 34–41.

[11] Y. Jia and M. Harman, *An analysis and survey of the development of mutation testing*, IEEE Transactions on Software Engineering 37(5), 2011, pp. 649–678.

[12] SCSC Assurance Case Working Group, *Goal Structuring Notation Community Standard, Version 3*, SCSC-141C, May 2021.

[13] T. Tao, *Mathematics in the age of AI*, preprint, 2026, arXiv.

[14] T. Tao, discussion of learning from studying a mathematical problem, Mastodon thread, 3 September 2026, opening post, accessed 5 September 2026.

[15] T. Tao, discussion of exploration and insight in the Navier–Stokes regularity problem, Mastodon thread, 3 September 2026, opening post; part 5 on preserving exploration, accessed 5 September 2026.

[16] Lean community, *Contributing to mathlib*, contributor guide, accessed August 2026.

[17] K. Yang et al., *LeanDojo: Theorem Proving with Retrieval-Augmented Language Models*, NeurIPS 2023.

[18] J. Storrs et al., *Pantograph: A Machine-to-Machine Interaction Interface for Advanced Theorem Proving, High Level Reasoning, and Data Extraction in Lean 4*, 2024, arXiv.

[19] M. Kripner and M. Straka, *OpenProver: Agentic and Interactive Theorem Proving with Lean 4*, 2026, arXiv.

[20] C. E. Brown, C. Kaliszyk, and J. Urban, *Agent Hunt: Bounty Based Collaborative Autoformalization With LLM Agents*, 2026, arXiv.

[21] Y. Zhang et al., *DreamProver: Evolving Transferable Lemma Libraries via a Wake–Sleep Theorem-Proving Agent*, 2026, arXiv.

[22] I. Takahara and T. Mizoguchi, *Toward Auditable AI Scientists: A Hypothesis Evolution Protocol for LLM Agents*, 2026, arXiv.

[23] D. Pratt, *Symposium: Trust via Auditable Records for Communities of AI Scientist Agents*, 2026, arXiv.

[24] A. Xin et al., *EurekAgent: Agent Environment Engineering is All You Need for Autonomous Scientific Discovery*, 2026, arXiv.